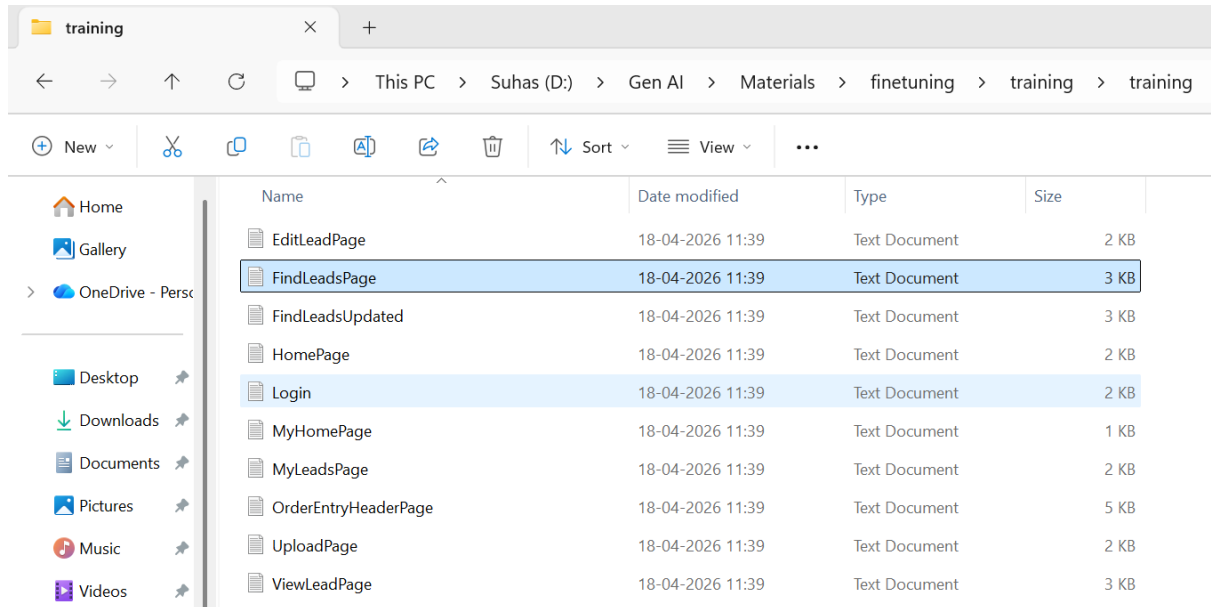


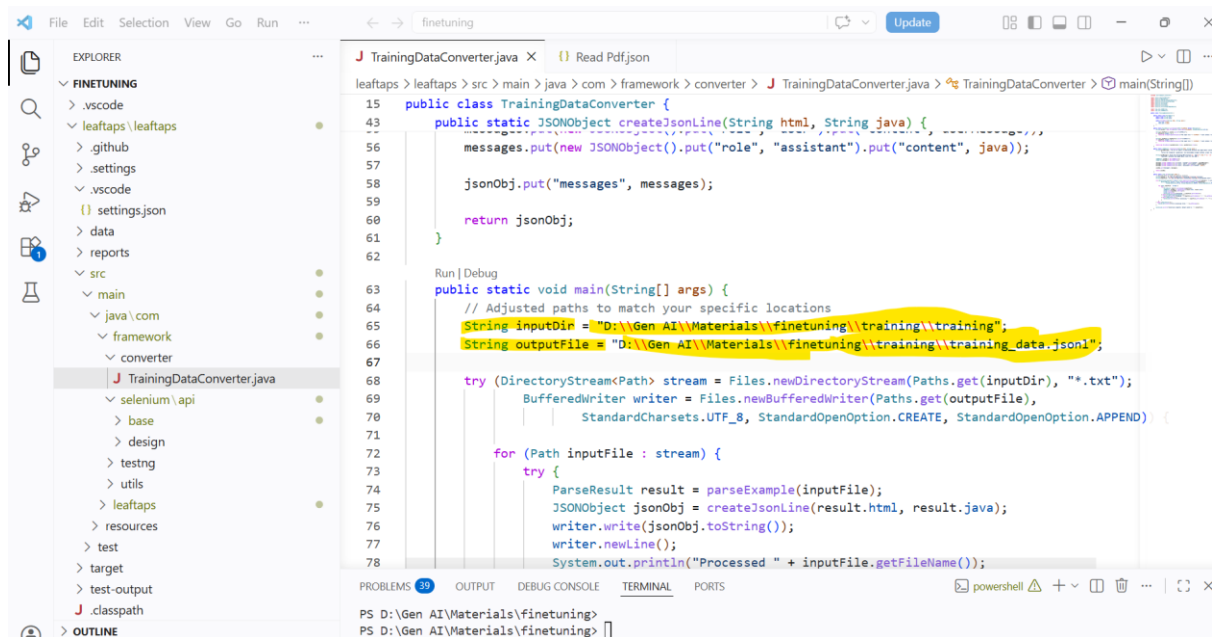
Fine Tuning

Preparation and Submission of json file:

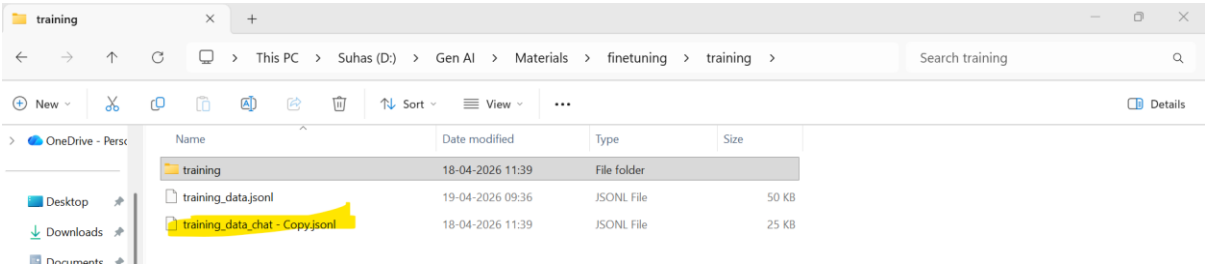
1. Input training data prepared with a minimum of 10 files. Each file containing a sample HTML prompt and a sample page object class for the respective page.



2. Input and Output path specified in TrainingDataConverter.java of the FineTuning Code. On running the code, we get the jsonl file generated in the specified O/P path as shown below.

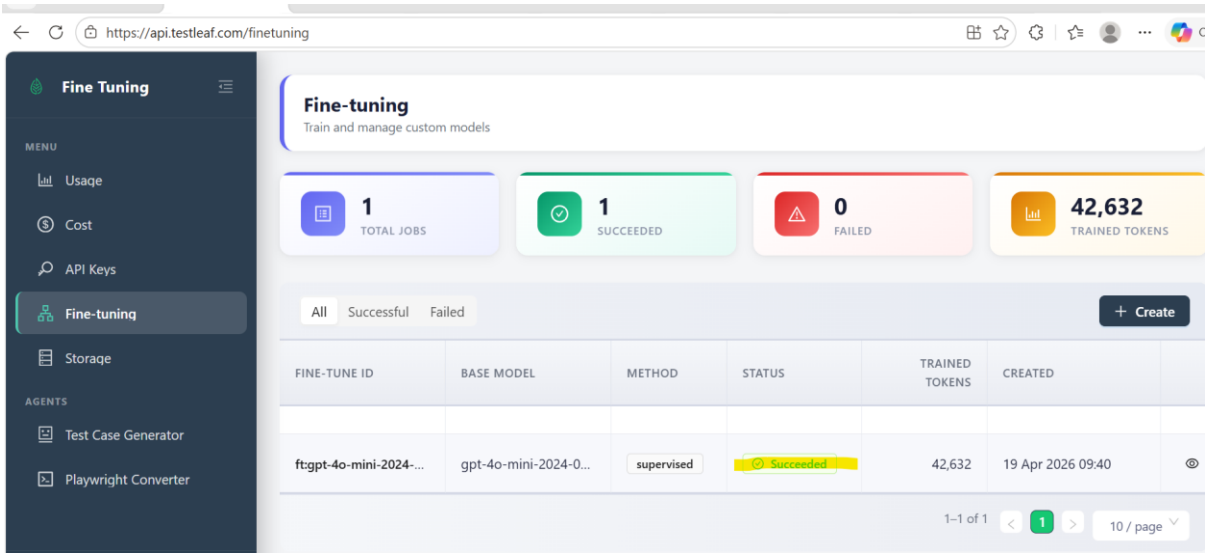


3. Generated jsonl file (File attached outside the folder, also attached as an object here)




training_data_chat -
Copy.jsonl

4. Fine-tuned using the above generated jsonl file, base model of gpt-4o-mini-2024-07-18 to create the new O/P model - ft:gpt-4o-mini-2024-07-18:testleaf-2::DWDuBlsU



ft:gpt-4o-mini-2024-07-18:testleaf-2::DWDuBlsU

Status

Succeeded

Job ID

ftjob-oMq513V7CedtKuCBFNz1mp3Z

Training Method

supervised

Base Model

gpt-4o-mini-2024-07-18

Output Model

ft:gpt-4o-mini-2024-07-18:testleaf-2::DWDuBlsU

Created At

19 Apr 2026 09:40:12

Trained Tokens

42,632

Epochs

4

Batch Size

1

LR Multiplier

0.21

Seed

1273606669

5. Added the new O/P model to pop.js of AI extension to have the test leaf fine-tuned model reflect in the chrome extension.

File Edit Selection View Go Run ...

ai-extension

Update

EXPLORER

AI-EXTENSION

ai-extension

assets/images

logo-small.png

lib

css

marked

marked.min.js

prism

src

config

appConfig.js

configUtils.js

content

content-script.js

content.js

scripts

api

groq-api.js

openai-api.js

testleaf-api.js

chat.js

init-colors.js

init-config.js

popups.js

prompts.js

tabs.js

styles

JS popup.js

ai-extension > src > scripts > JS popup.js > document.addEventListener('DOMContentLoaded') callback > modelsByProvider > testleaf

```
5  nt.addEventListener('DOMContentLoaded', async () => {
6    if (!chrome.storage) {
97    console.error('Chrome storage is not available');
98    return;
99  }
100
101  nst providerSelect = document.getElementById('providerSelect');
102  nst modelSelect = document.getElementById('modelSelect');
103  nst groqApiKeyInput = document.getElementById('groqApiKey');
104  nst openaiApiKeyInput = document.getElementById('openaiApiKey');
105  nst testleafApiKeyInput = document.getElementById('testleafApiKey');
106
107  Model options by provider
108  nst modelsByProvider = {
109    groq: [
110      { value: 'meta-llama/llama-4-maverick-17b-128e-instruct', label: 'meta-llama/llama-4-maverick-17b-128e-instruct' },
111      { value: 'openai/gpt-oss-120b', label: 'openai/gpt-oss-120b' },
112    ],
113    openai: [
114      { value: 'gpt-4o', label: 'GPT-4o' },
115      { value: 'gpt-4o-mini', label: 'GPT-4o Mini' },
116    ],
117    testleaf: [
118      { value: 'ft:gpt-4o-mini-2024-07-18:testleaf-2::DWDuBlsU', label: 'TestLeaf FineTuned GPT-4o Mini' },
119    ],
120  ];
121
122  Function to update model options based on selected provider
```

Extension: (Model Selected)

 AI Code Generator  

Code Generator

Settings

Language Binding

Java

Browser Engine

Selenium

Select LLM Provider

Testleaf

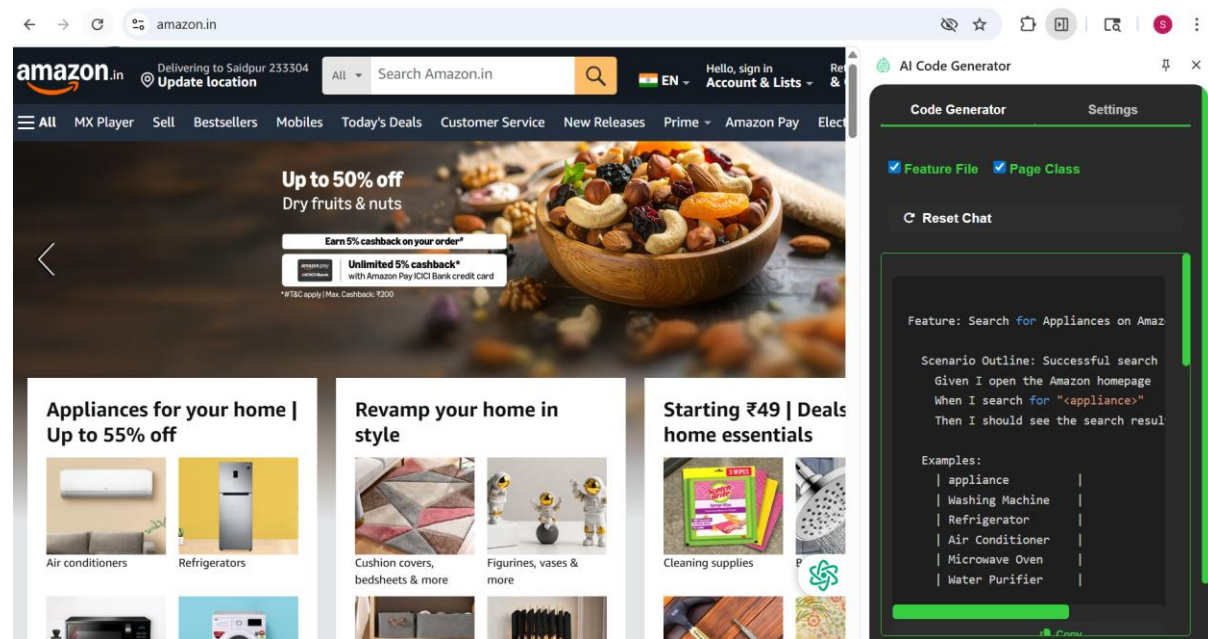
Select Model

TestLeaf FineTuned GPT-4o Mini

Testleaf API Key

.....

Code Generated:



Feature: Search for Appliances on Amazon

Scenario Outline: Successful search for appliances

Given I open the Amazon homepage

When I search for "<appliance>"

Then I should see the search results for "<appliance>"

Examples:

appliance	
Washing Machine	
Refrigerator	
Air Conditioner	
Microwave Oven	
Water Purifier	

```
package com.leafyaps.stepdefs;

import io.cucumber.java.en.*;
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.*;

public class SearchStepDefinitions {

    private WebDriver driver;

    private WebDriverWait wait;

    @io.cucumber.java.Before
    public void setUp() {

        System.setProperty("webdriver.chrome.driver", "path/to/chromedriver");

        driver = new ChromeDriver();

        wait = new WebDriverWait(driver, Duration.ofSeconds(10));

        driver.manage().window().maximize();

    }

    @io.cucumber.java.After
    public void tearDown() {

        if (driver != null) driver.quit();

    }

    @Given("I open the Amazon homepage")
    public void openAmazonHomepage() {

        driver.get("https://www.amazon.in/");

    }

}
```

```

@When("I search for {string}")

public void searchForAppliance(String appliance) {

    WebElement searchBox =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("twotabsearchtextbox")))
;

    searchBox.sendKeys(appliance);

    searchBox.sendKeys(Keys.RETURN);

}

@Then("I should see the search results for {string}")

public void verifySearchResults(String appliance) {

    WebElement resultsHeader =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.xpath("//span[contains(text
()),'results for']"))));

    String resultsText = resultsHeader.getText();

    assert resultsText.contains(appliance) : "Expected to see results for " + appliance + "
but found: " + resultsText;

}

}

```